

L07 Registration and map representations

Align every scan into one frame, then decide what the map is made of.

Registration and map representations

L07 · Unit 1

*Before a robot can build a map it has to put every scan in one frame.
Then it has to decide what the map is made of.*

Registration aligns two scans. A volumetric map fuses the aligned scans into a grid of distances. A neural map fits a network to the same data and answers questions by evaluating it.

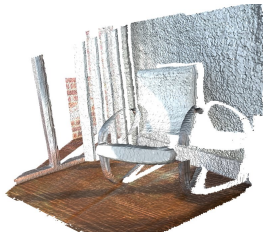


Figure: Open3D RGB-D integration tutorial, MIT License

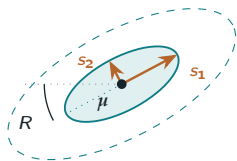
Today

1. **The rest of Monday:** 3D Gaussians, the catalog side by side, and the networks that read point sets.
2. **Registration:** alignment from matched points, ICP, point-to-plane, robust costs, global registration.
3. **Volumetric maps:** KinectFusion, the TSDF and its weights, Voxblox, and the ESDF a planner reads.
4. **Neural maps:** NeRF and volume rendering, positional encoding, Instant-NGP, iMAP and NICE-SLAM.
5. **The comparison:** drift, memory, and what it costs to ask each map a question.

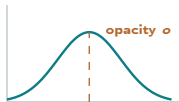
Reading: Voxblox, with NeRF, Instant-NGP and NICE-SLAM. Monday's PointNet papers close out in the first third.

3D Gaussians

1 of 5: one Gaussian, from Monday



solid: one standard deviation; dashed: two



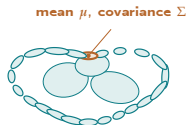
light blocked, along the long axis

$$G(x) = \exp \left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu) \right)$$

- **Mean μ :** where it sits.
- **Covariance Σ :** its shape and orientation, stored as a rotation R and three scales s_1, s_2, s_3 .
- **Opacity o :** how much light it blocks at its centre; at x it blocks $o G(x)$.
- **Colour:** three numbers, or more so it can change with viewing direction.

3D Gaussians

2 of 5: a scene is millions of them



flat along the surface, large where nothing varies

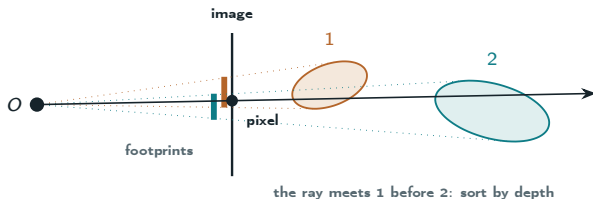
- A map fitted to photographs of one room holds a few million Gaussians.
- Thin, flat ones line up along surfaces; large round ones fill regions where nothing varies.
- Nothing connects them: no grid and no mesh.



Figure: Kerbl et al., 3D Gaussian Splatting (2023), CC BY 4.0

3D Gaussians

3 of 5: drawing an image



1. **Project** each Gaussian through the camera. A 3D ellipsoid lands on the image as a 2D ellipse, its **footprint**: L01's pinhole, applied to a blob.
2. **Sort** the Gaussians by depth.
3. **Blend** at every pixel, front to back. Each Gaussian takes its share of the light that is still left.

3D Gaussians

4 of 5: one pixel, by numbers

At this pixel, Gaussian 1 blocks $\alpha_1 = 0.6$ of the light and Gaussian 2 blocks $\alpha_2 = 0.5$. Each α_i is the opacity times the footprint's value at the pixel.

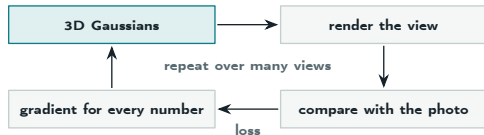
1. Gaussian 1 takes 0.6 of the pixel, and $1 - 0.6 = 0.4$ of the light gets past it.
2. Gaussian 2 takes 0.5 of what is left: $0.4 \times 0.5 = 0.2$.
3. $0.4 \times (1 - 0.5) = 0.2$ is left for whatever is behind.



$$C = \sum_i c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) = 0.6 c_1 + 0.2 c_2 + \dots$$

3D Gaussians

5 of 5: *what the map is for*

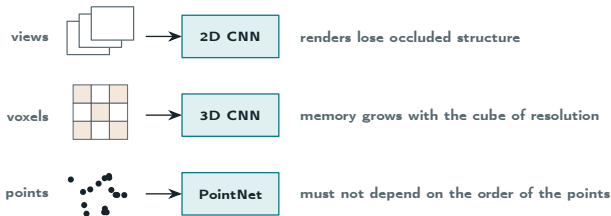


- **Fitting:** every step of the rendering is differentiable, so gradient descent (L05) adjusts every Gaussian until the renders match the photographs.
- **Good for:** new views in real time, and tracking a camera by comparing what it sees with a render.
- **Not for:** free space, surfaces or distances. A planner needs another representation beside it.
- L09 builds SLAM on Gaussians.

The catalog, side by side

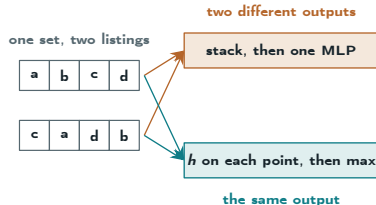
	Memory	Free space	Distance query	Online update	Typical use
Point cloud	per point	no	neighbour search	append	raw sensing, registration
Voxel grid	volume	yes	lookup	cheap	small workspaces, 3D CNNs
Octree	roughly the surface	yes, and unknown	$O(\text{depth})$ lookup	cheap	occupancy maps, exploration
TSDF, ESDF	volume, or blocks near surfaces	yes	lookup	weighted average	fusion, collision checking, planning
Mesh	surface	no	geometric search	awkward	rendering, simulation, grasping
Implicit network	the weights	learned	forward pass	retraining	shape completion
3D Gaussians	per Gaussian	no	not directly	optimisation	view synthesis, photometric SLAM

Learning directly on 3D data



- Rendered views reuse 2D CNNs but hide what is occluded, and a per-pixel answer must be carried back to 3D.
- Voxel grids let convolution work in 3D at cubic cost: 64^3 is 262,144 cells, 256^3 is 16.8 million.
- **PointNet** (Qi et al., CVPR 2017) reads the points themselves. The price is that the network must accept a set.

A function of a set



- n points can be listed in $n!$ orders; 1024 points in about 10^{2639} . The answer must be the same for all of them.
- It should also survive a few missing or extra points.

Why stacking fails, and a form that cannot fail

Two points, stacked: $W \begin{bmatrix} a \\ b \end{bmatrix} = W_1 a + W_2 b$, but $W \begin{bmatrix} b \\ a \end{bmatrix} = W_1 b + W_2 a$. Equal for every input only if $W_1 = W_2$.

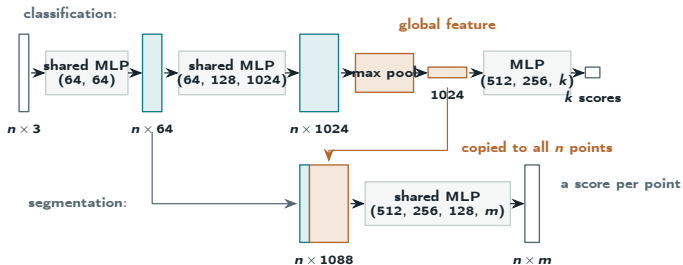
Sum and max treat every point alike:
 $\max(h(a), h(b)) = \max(h(b), h(a))$.

$$f(\{x_1, \dots, x_n\}) \approx \gamma\left(\max_{i=1, \dots, n} h(x_i)\right)$$

h and γ are MLPs; the max is taken separately in each feature.

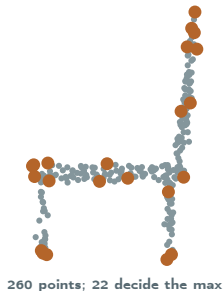
PointNet

Qi, Su, Mo and Guibas, CVPR 2017



- **Shared MLP:** the same weights on every point, applied to each alone, so n can vary.
- **Max pool:** over the n points, in each of the 1024 features: the only step where points meet.
- **Segmentation:** a point's 64 local features plus the 1024 global ones give local and scene context.

Critical points



- The max keeps, for each of the 1024 features, the one point that maximises it. At most 1024 points decide the global feature: the **critical points**.
- Remove any other point, or add one that raises no maximum, and the output does not change. With half the points missing, the paper's accuracy drops by only 2 to 4 points.
- The paper's plots of critical points look like each object's skeleton and outline: a sparse summary of the shape.

PointNet++: set abstraction

Qi, Yi, Su and Guibas, *NeurIPS 2017*

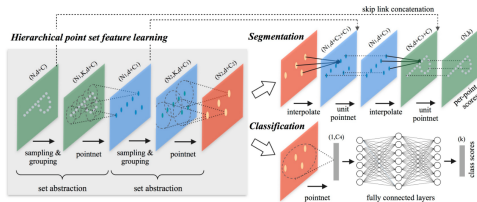
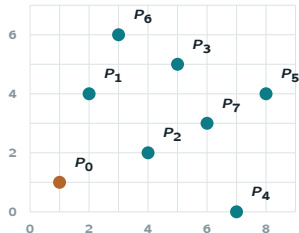


Figure: Qi et al., PointNet++ repository, MIT License

- **Set abstraction:** *sample* centres by farthest point sampling, *group* the K points within radius r of each, and run a small **PointNet** on each group in local coordinates. Stack it: fewer points, more features, wider regions.
- For segmentation, interpolate the coarse features back to every point: L05's encoder-decoder, on point sets.

Farthest point sampling, by hand



Each new sample is the point farthest from every sample so far. It covers the cloud evenly; random sampling piles samples where points are dense.

■ Start from P_0 and pick three more

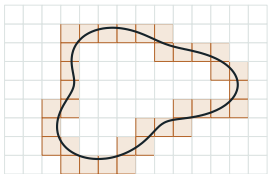
Min squared distance to $\{P_0\}$ for the rest:
10, 10, 32, 37, 58, 29, 29. Largest: P_5 (58).

Min squared distance to $\{P_0, P_5\}$ for the rest:
10, 10, 10, 17, 29, 5. Largest: P_6 (29).

Min squared distance to $\{P_0, P_5, P_6\}$ for the rest:
5, 10, 5, 17, 5. Largest: P_4 (17).

Samples in order: P_0, P_5, P_6, P_4 .

Most of a 3D grid is empty



34 of 126 cells occupied; all 126 stored

Surfaces are two-dimensional. A scan fills a thin shell of the grid, a few percent of the voxels or less.

■ One $3 \times 3 \times 3$ convolution, 64 channels in and out, on Monday's room at 5 cm, with 60,000 of 2,400,000 voxels occupied

$27 \cdot 64 \cdot 64 = 110,592$ multiply-adds per output voxel

dense: 2.7×10^{11} , sparse: 6.6×10^9

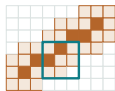
40 times less work, and the gap widens as the voxels get finer: the occupied shell grows with area, the grid with volume.

Sparse convolution

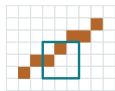
Choy, Gwak and Savarese, CVPR 2019

$$x_u^{\text{out}} = \sum_{i \in \mathcal{N}(u, C^{\text{in}})} W_i x_{u+i}^{\text{in}} \quad \text{for } u \in C^{\text{out}}$$

regular sparse convolution manifold sparse conv



35 active outputs



7 active outputs

- Sum only over kernel offsets whose input site is occupied; output only at the sites in C^{out} .
- **Regular:** output wherever the kernel touches an input, so the active set dilates. **Submanifold:** keep $C^{\text{out}} = C^{\text{in}}$.
- Stride 2 merges sites into a coarser grid: pooling's role in a 2D CNN.

MinkowskiNet

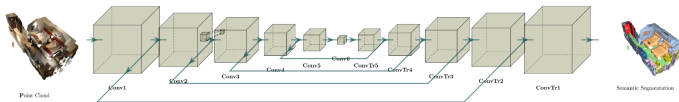
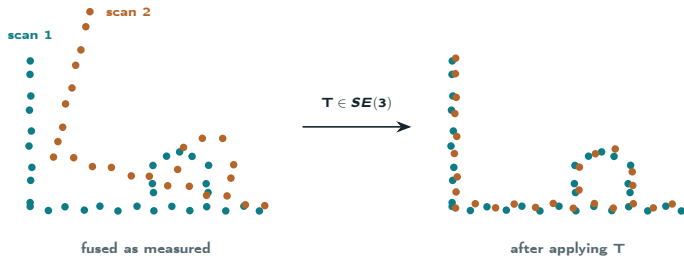


Figure: Minkowski Engine documentation, MIT License

- **MinkUNet** is a U-Net of sparse convolutions: strided ones down, transposed ones up, skip connections across. It is L05's encoder-decoder, on a sparse grid, and it still backs LiDAR segmentation and 3D detection.
- The Minkowski Engine implements the same operator in any dimension, including 4D space-time for sequences of scans.
- **Points or voxels:** PointNet++ keeps exact coordinates and pays for a neighbour search at every layer; sparse convolution quantises to voxels and gets a hash lookup and a weight per offset. Attention over neighbours arrives in L10.

Two scans, one map

registration



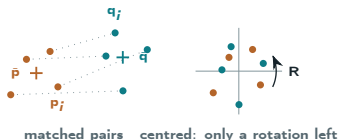
- Each scan is measured in the sensor frame at the moment it was taken. Fusing two needs the pose between them, $T \in SE(3)$: six numbers.
- Odometry gives a guess. **Registration** refines it from the geometry the two scans share.
- Fuse before aligning and every wall appears twice.

If you knew the correspondences

Matched pairs (p_i, q_i) , from features (L04) or a target. Choosing R and t to minimise $\sum_i \|Rp_i + t - q_i\|^2$ needs no iteration: one SVD of a 3×3 matrix answers it.

- **Translation first:** it enters linearly, so $t = \bar{q} - R\bar{p}$.
Centre both sets and only the rotation is left.
- **Rotation:** with $H = \sum_i p'_i q'_i{}^\top$, minimising the sum means maximising $\text{tr}(RH)$:
- **Trap:** without the determinant term the SVD can return a reflection, $\det R = -1$, on noisy or nearly coplanar points.

$$H = U\Sigma V^\top, \quad R = V \text{diag}(1, 1, \det(VU^\top)) U^\top$$

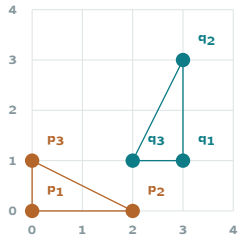


A worked example in 2D

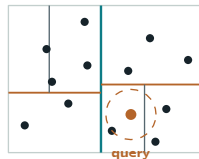
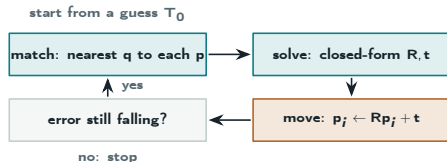
Three pairs: $p = (0, 0), (2, 0), (0, 1)$ and $q = (3, 1), (3, 3), (2, 1)$. In 2D, $\text{tr}(\text{RH}) = \cos \theta (H_{11} + H_{22}) + \sin \theta (H_{12} - H_{21})$, so the angle comes straight out of H .

$$\bar{p} = \left(\frac{2}{3}, \frac{1}{3}\right), \quad \bar{q} = \left(\frac{8}{3}, \frac{5}{3}\right), \quad H = \frac{1}{3} \begin{bmatrix} 2 & 8 \\ -2 & -2 \end{bmatrix}$$

$$\theta = \text{atan2}\left(\frac{10}{3}, 0\right) = 90^\circ, \quad t = \left(\frac{8}{3}, \frac{5}{3}\right) - \left(-\frac{1}{3}, \frac{2}{3}\right) = (3, 1)$$



ICP: guess the matches, then solve



- **Iterative closest point** (Besl and McKay, 1992): match each source point to its nearest target point, solve in closed form, move, repeat.
- Nearest-neighbour search is most of the cost. A **k-d tree** splits the target at the median along alternating axes, so a query takes about $\log N$ steps.
- No iteration can increase the error, so ICP converges, but only to the nearest local minimum.

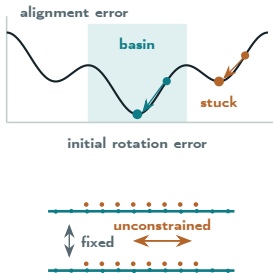
Point-to-plane



$$E(T) = \sum_i \left((T p_i - q_i)^\top n_i \right)^2$$

- Chen and Medioni (1992): charge the distance to the tangent plane at the match, with n_i from the target's neighbours.
- Two scans never sample a wall at the same spots. Point-to-point pulls each point to a sample; point-to-plane lets it slide along the surface.
- Far fewer iterations on scenes made of surfaces: fitness 0.62 after 30, where point-to-point manages 0.37.

When ICP fails, and what to do

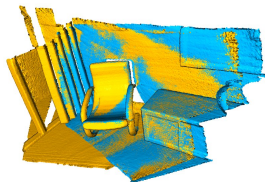


- **A bad start:** ICP slides into the nearest minimum. **Symmetry:** a cylinder, or identical shelves. **Degeneracy:** a corridor is free along its length. **Little overlap:** most nearest neighbours are wrong.
- **Gate:** drop pairs farther apart than a threshold, 2 cm in the Open3D example. **Trim:** keep the best 70–90% by residual.
- **Robust cost:** replace r^2 by $\rho(r)$, Huber or Tukey, and reweight each row by $w(r_i)$ each iteration.

Global registration, then ICP

Open3D tutorial

- With no initial guess, match descriptors instead of positions. **FPFH** (Rusu et al., 2009) summarises the angles between normals around a point as a 33-bin histogram, unchanged by pose.
- Three matched pairs fix a rigid motion. RANSAC samples triples, solves with the SVD, and counts the pairs within a threshold: with L04's count, $w = 0.5$ needs 35; $w = 0.2$ needs 574; $w = 0.1$ needs 4,603.
- The result is coarse by design: ICP on the full cloud makes it tight.

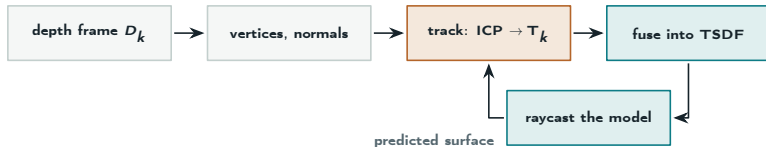


RANSAC on downsampled clouds, then
point-to-plane ICP

Figure: Open3D tutorial, MIT License

KinectFusion

volumetric maps



- Newcombe et al. (2011): a handheld depth camera, a GPU, and a dense surface built live at 30 Hz.
- The map is a **TSDF** in a fixed cube, 512^3 voxels over a few metres. Each frame is tracked by point-to-plane ICP against a surface raycast from the map, so errors do not pile up frame after frame.
- Voxblox keeps the TSDF and changes how it is stored, weighted and used.

Fusing many measurements

the TSDF from L06, one voxel at a time

$$D \leftarrow \frac{WD + w d}{W + w}, \quad W \leftarrow \min(W + w, W_{\max})$$

■ One voxel, three frames, $w = 1$ each: $d = +5, -1, +5$ cm

$$D_1 = +5, W_1 = 1 \quad D_2 = \frac{1 \cdot 5 + (-1)}{2} = 2, W_2 = 2$$

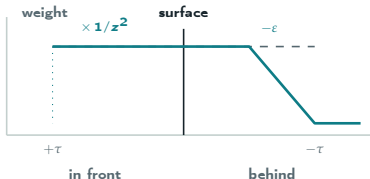
$$D_3 = \frac{2 \cdot 2 + 5}{3} = 3, W_3 = 3$$

All in centimetres.

- Each voxel on a ray stores its distance to the surface *along the ray*, truncated at $\pm\tau$ (L06). Fusing is a running weighted average: the voxel keeps D and W , and nothing about the frames that made them. Noise averages out.

Which measurements to trust

weighting



■ The same three frames with $w = 1/z^2$, at $z = 1, 2, 1$ m

$$w = 1, 0.25, 1 : \quad D_2 = \frac{5 + 0.25(-1)}{1.25} = 3.8$$

$$D_3 = \frac{1.25(3.8) + 5}{2.25} = 4.33$$

The far frame counts a quarter.

- **KinectFusion:** $w = 1$ always. **Voxblox:** $w = 1/z^2$, because depth noise grows with range (L04), times a drop-off behind the surface so the back of a thin object is not carved away.

Voxblox: blocks and a hash map

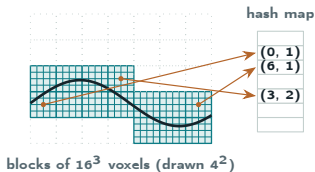


Image: Voxblox README, ETH Zurich ASL, BSD-3-Clause

- Voxels come in **blocks** of 16^3 , allocated only where rays land and found through a hash map on the block index. The map grows with the robot instead of living in a fixed cube.
- **Incremental meshing:** marching cubes reruns only on blocks that changed. The holes in the mesh are voxels no ray reached: unknown, and a planner must not treat unknown as free.

Building the ESDF by brushfire

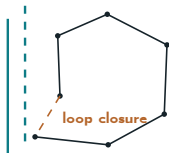


- Obstacle cells start at 0. Every other cell takes the smallest value among its neighbours plus the step to it, 1 sideways or 1.4 on the diagonal, and the wavefront spreads outward: Dijkstra on the grid.
- Voxblox seeds it from the TSDF voxels near the zero crossing and updates only what a new scan changed.
- The dashed robot of radius 1.5 cells sits where $d = 2.4$, so it is clear.
- Two queues: one for distances that shrink as obstacles appear, one for distances that grow as they go away.

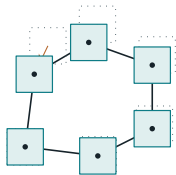
A map that cannot bend

drift

wall fused twice



one TSDf: stuck



submaps: each moves with its pose

- Every voxel was fused with the pose known at the time. When a loop closure later corrects those poses, the fused distances cannot follow: the TSDf keeps no record of which frame wrote what.
- **Submaps:** fuse short stretches into separate TSDFs, one per pose, and move each rigidly after a loop closure (Voxgraph, 2020). BundleFusion (2017) de-integrates and re-integrates frames instead.

NeRF: a scene as a function

neural maps

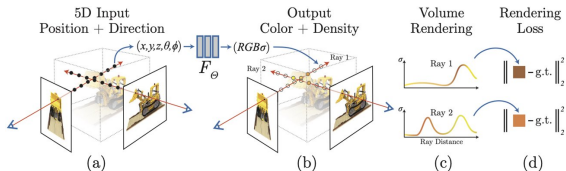


Figure: Mildenhall et al., NeRF repository, MIT License

- Mildenhall et al. (2020): an MLP $F_\theta : (x, d) \mapsto (c, \sigma)$ takes a 3D position and a viewing direction and returns a colour and a **density** σ , the rate at which a ray is absorbed there.
- It is trained on posed photographs. The poses come from structure from motion (COLMAP in the paper): L04's pipeline, run first.
- A pixel is rendered by sampling points along its ray and compositing them, and the loss compares rendered pixels with real ones.

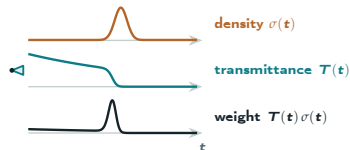
Volume rendering

$$T(t) = \exp\left(-\int_0^t \sigma(s) ds\right), \quad C = \int_0^\infty T \sigma c dt$$

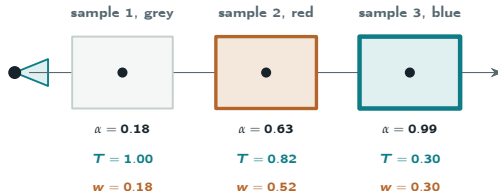
On a segment of length δ_i with constant σ_i :
 $\alpha_i = 1 - e^{-\sigma_i \delta_i}$.

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j), \quad \hat{C} = \sum_i T_i \alpha_i c_i = \sum_i w_i c_i$$

T is the chance nothing has stopped the ray yet: this morning's Gaussian blending sum, with α from a density. Every step is differentiable.



Three samples by hand



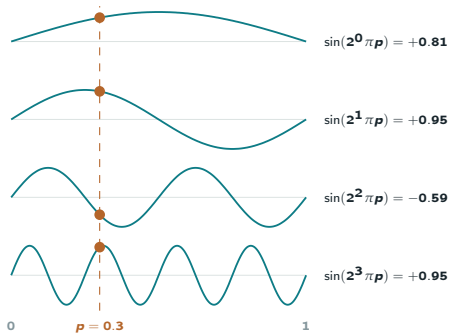
■ $\delta = 0.5$ and $\sigma = 0.4, 2, 10$: **opacities, transmittance, weights, colour**

$$\alpha = 1 - e^{-\sigma\delta} = 0.18, 0.63, 0.99, \quad T = 1.00, 0.82, 0.30, \quad w = T\alpha = 0.18, 0.52, 0.30$$

$$\hat{C} = 0.18 c_{\text{grey}} + 0.52 c_{\text{red}} + 0.30 c_{\text{blue}} = (0.61, 0.09, 0.39)$$

The weights sum to 0.998. The blue sample is almost opaque, but only 30% of the light reaches it.

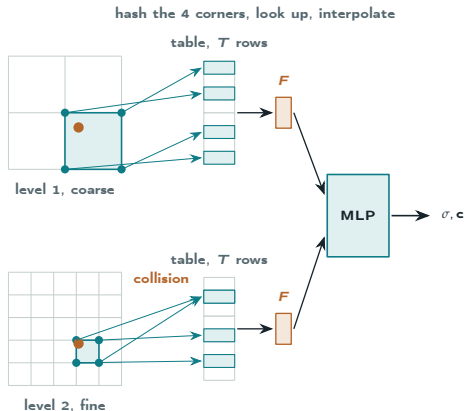
Positional encoding



$$\gamma(p) = (\sin(2^0 \pi p), \cos(2^0 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p))$$

- Given raw coordinates, an MLP learns smooth functions first and fine detail slowly: **spectral bias**. NeRF without the encoding renders a blurred scene.
- Encode each coordinate before the MLP.
 $L = 10$ for position gives 60 numbers;
 $L = 4$ for direction gives 24.

Instant-NGP: a hash table of features



- Müller et al. (2022): L grids of increasing resolution. At each level, hash the corners of the cell around x into a table of T learned vectors of length F , and interpolate.
- Concatenate the L vectors into a tiny MLP. Most of the model is now the tables, and a query reads only $8L$ rows.
- Collisions are not resolved. Gradients from surfaces dominate a shared row, and the other levels separate the rest.

Neural maps that run online

iMAP and NICE-SLAM

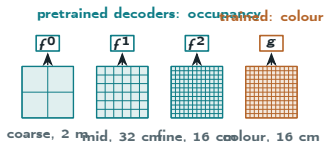


Image: NICE-SLAM repository, Apache-2.0

- **iMAP** (Sucar et al., 2021): the map is one MLP, trained live from RGB-D. Tracking optimises the pose on $SE(3)$ through the renderer with the network frozen; mapping trains the network and the keyframe poses.
- **NICE-SLAM** (Zhu et al., 2022) puts features in coarse, mid and fine grids with pretrained decoders. A frame updates only the cells its rays cross, so the map scales to an apartment.

Two ways to hold a map

	Voxblox: TSDF and ESDF	NeRF, iMAP, NICE-SLAM
Query a point	index a voxel: one memory read	interpolate features, run an MLP
Free space	observed free, near a surface, or unknown	whatever the network predicts, seen or not
Unseen regions	holes	filled in, plausibly or not
Appearance	one colour per voxel	new views, view-dependent colour
For a planner	ESDF distance and gradient	not stored: derive it or sample it
Loop closure	submaps, or re-integrate	none in iMAP or NICE-SLAM
Compute	CPU, at camera rate	GPU, usually below camera rate



Image: Voxblox README, ETH Zurich ASL,
BSD-3-Clause

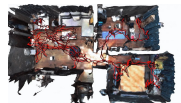


Image: NICE-SLAM repository, Apache-2.0

What you should be able to do

- Say why a network on points needs a symmetric function, and trace PointNet's tensor shapes.
- Run farthest point sampling by hand, and say what a sparse convolution skips.
- Compute the rigid motion between matched point sets with centroids and an SVD, and say what the determinant term is for.
- Say when point-to-plane beats point-to-point, and name three ways ICP fails.
- Update a TSDF voxel with a weighted average, and grow an ESDF outward from the obstacles.
- Composite samples along a ray into a colour with the volume rendering weights.

To go further: the Open3D tutorials on ICP, global registration and RGB-D integration; the Voxblox documentation; the nerfstudio pages on NeRF and Instant-NGP.

Next

L08 guest lecture

- **Sun Sep 27:** both diagnostic quizzes are due.
- **Mon Sep 28, L08:** Wen Jiang (Waymo) on active perception: choosing the next view with Fisher information over a radiance field. The response paper is **FisherRF**, due before class.
- **Wed Sep 30:** Seminar I, student-led, on the Unit 1 papers.

FisherRF builds on today's volume rendering.